COMP4135 SEM Final Group Report

Group 2 (Tactical Cards)

Team members

Guangjun HU (20511787)
Hongshuo HU (20808603)
Cheng YE (20513323)
Zehan CHEN (20807750)

School of Computer Science
University of Nottingham

Submission Date: TBC - confirm on Moodle

Abstract

Tactical Cards is a Unity-based tactical card combat prototype designed for teenagers and casual strategy players who benefit from readable, small-scope decision making. The final project delivers a complete single-battle loop with a title screen, a playable tactical encounter on a grid board, a result screen, card-driven unit actions, deterministic enemy response, and a themed runtime HUD. Development was deliberately kept within a compact vertical slice so the team could prioritise maintainable code structure, automated validation, and visible teamwork evidence alongside gameplay implementation. The delivered project combines early requirements work from WR1, implementation and scope-control decisions from WR2, and a validation-heavy WR3 plus post-WR3 polish cycle. By the end of the project, the codebase had stable EditMode and PlayMode test coverage, a documented issue-closure trail, and a clearer UI/UX layer that addressed earlier feedback about readability, feedback, and onboarding.

Contents

Abstract **1**

1 Introduction **5**
1.1 Motivation 5
1.2 Aims and Objectives 5
1.3 Description of the Work 6

2 Background and Related Work **7**
2.1 Design Context 7
2.2 Reference Strategy 8
2.3 Related-Work Boundary 8

3 Requirements Engineering **9**
3.1 Requirements Elicitation 9
3.2 Requirements Specification 9
3.3 Requirements Validation 11

4 Development **12**
4.1 System Design 12
4.2 Implementation Strategics 14
4.3 Simulation Design 15
4.4 UI Design 16

5 Test & System Evaluation **18**
5.1 Test Strategy 18
5.2 Test Methods 19
5.3 Code Coverage 20
5.4 Evaluation 21

6 Evolution & Maintenance **23**
6.1 Delivered Maintainability Measures 23
6.2 Deferred Work and Future Evolution 24

7 Problem & Reflections **25**
7.1 Technical Problem 25
7.2 Project Management 26
7.3 Team Management 26

8 Conclusion **28**

Reference 29

A User-manual 30
A.1 Purpose . . . 30
A.2 Opening the Project . . . 30
A.3 Starting a Battle . . . 30
A.4 Core Interaction Flow . . . 30
A.5 Reading the Battle HUD . . . 31
A.5.1 Top Bar . . . 31
A.5.2 Left Panel . . . 31
A.5.3 Bottom Hand Panel . . . 31
A.5.4 World-space Unit Feedback . . . 31
A.6 Card Reference . . . 31
A.7 Understanding the Turn Loop . . . 32
A.8 Ending the Battle . . . 32
A.9 Troubleshooting . . . 32

B Meeting Minutes 33
B.1 Meeting Record Scope . . . 33
B.2 Recovered Implementation Checkpoint . . . 33
B.2.1 Purpose . . . 33
B.2.2 Recovered Decisions . . . 33
B.2.3 Recovered Follow-up Tasks . . . 34
B.3 WR1 Kickoff Meeting . . . 34
B.3.1 Context . . . 34
B.3.2 Decisions . . . 34
B.3.3 Follow-up . . . 34
B.4 WR1 Alignment Meeting . . . 34
B.4.1 Context . . . 34
B.4.2 Decisions . . . 34
B.4.3 Follow-up . . . 35
B.5 Testing and Validation Meeting . . . 35
B.5.1 Attendance . . . 35
B.5.2 Agenda . . . 35
B.5.3 Progress Since Last Meeting . . . 35
B.5.4 Key Discussion Points . . . 35
B.5.5 Decisions Made . . . 36
B.5.6 Action Items . . . 36
B.5.7 Issue Closure Status . . . 36
B.5.8 Next Meeting . . . 36
B.6 Current Gap Note . . . 36

C Test Case 37
C.1 Code Coverage . . . 37
C.2 Test Coverage . . . 37
C.3 Test Results . . . 37

D Development Planning 38

E Version Control **39**

F Docs **40**

Chapter 1

Introduction

Tactical Cards is the final Unity coursework project developed by Group 2 for the COMP4135 Software Engineering Management module. The project answers the CW1 brief by producing a working game suitable for teenagers while also demonstrating requirements engineering, implementation planning, validation, documentation, and team-based software delivery [9, 10]. Rather than attempting a content-heavy RPG or a large collectible card game, the team chose a compact tactical card battle that could be implemented, tested, and refined within the available coursework window.

1.1 Motivation

The team's starting motivation was to build a game that encouraged planning, positioning, and short-horizon tactical decision making without overwhelming the player with large maps, progression systems, or complex rules. This aligned well with the coursework requirement to produce a game for teenagers: the design could stay strategically meaningful while remaining readable, approachable, and demonstrable in a short play session [5, 9].

The tactical card format was chosen because it combined two strengths. First, a grid-based battle creates visible positional decisions that are easy to explain and verify. Second, a limited hand of command cards introduces turn-by-turn choice without requiring a large content set. This combination allowed the team to aim for a game that felt deliberate rather than random, while still keeping the scope realistic for a small student team.

1.2 Aims and Objectives

The project had four main objectives.

1. Deliver a complete playable battle loop in Unity, including player turn, enemy turn, and battle end state.
2. Keep the gameplay small enough to be implemented and validated properly within coursework scope.
3. Build the game in a way that leaves clear evidence of engineering practice, including requirements traceability, testing records, and version-control

workflow.

4. Improve readability and feedback so a first-time player can understand what to do without external explanation.

Success was therefore defined less by content quantity than by the completeness and quality of one polished vertical slice. A successful outcome meant that the player could enter the game, start a battle, understand the core interaction model, finish the encounter, and see the result in a stable and test-backed system.

1.3 Description of the Work

The final system is a compact tactical battle demo built around one encounter scene. The player begins from a title screen, starts a battle, controls two allied units on a small grid, uses a hand of tactical cards to move, attack, defend, reposition, or recover, and then ends the turn so enemy units can act. The battle finishes in a clear victory or defeat state and can then be replayed or exited back to the menu.

Over time, the project evolved from a prototype-focused slice into a more complete coursework deliverable. WR1 established the concept and initial requirements. WR2 locked down the main implementation decisions, including the single-battle vertical slice and explicit, readable battle logic. WR3 concentrated on test strategy and execution evidence. After WR3, the team extended the front-end flow, HUD feedback, UI theming, and battle readability so the final project would feel complete enough for final submission rather than only technically functional.

The result is intentionally modest in content, but it is coherent in scope: one polished encounter, one maintainable codebase, one testable battle loop, and one reportable development trail.

Chapter 2

Background and Related Work

The delivered game sits in a recognisable design space between a small-scale tactical skirmish game and a card-driven command system. However, the project was never intended to compete with content-heavy strategy RPGs or large collectible card games. From the start, the team treated related work as a guide for framing interaction patterns rather than a source of gameplay scope inflation. That position is consistent with both the coursework brief and the early one-pager, which emphasised a readable, teenager-friendly tactical prototype rather than a broad commercial feature set [9, 5].

2.1 Design Context

The core design context was a lightweight tactical grid battle in which the player controls a very small team and spends a limited hand of cards each turn. This places Tactical Cards closer to a command-oriented tactics prototype than to a deckbuilding game. Positioning, range, targeting, and action ordering are the main decision variables, while content breadth is intentionally constrained. The project therefore uses cards as a readable action language, not as a collectible system with large pools, rarity, or synergy chains [5].

Unity was selected because the game depends on both world-space interaction and screen-space feedback. The board, tile highlights, unit placement, and combat feedback all live in the same runtime environment as the HUD, menu flow, and result presentation. That makes Unity a better fit than a document-style prototype or a purely web-first interface for this coursework. It also allowed the team to demonstrate software-engineering concerns that matter for game development in practice, such as separating turn rules from view state, isolating assets, and validating behaviour across both scene logic and UI flow.

Just as importantly, the team chose to keep the product as one polished encounter rather than stretching toward multiple maps, progression systems, or a larger content pipeline. This was not simply a feature cut; it was a design decision to preserve clarity and testability. A compact battle loop gave the team enough room to implement and validate selection order, card resolution, enemy response, battle-end transitions, and usability refinements without diluting the engineering evidence with unfinished systems [9].

2.2 Reference Strategy

The team used external references selectively and at two different levels. At the gameplay level, the project borrowed only broad genre patterns: grid occupancy, ranged versus melee positioning, and card-labelled actions. No external gameplay framework was imported into the runtime. The battle loop, flow coordination, grid logic, enemy response, and HUD behaviour were authored within the project codebase under the main runtime scripts, which keeps ownership boundaries clear and makes the resulting software easier to explain in the report.

At the asset level, the team deliberately adopted a 'CC0-first' strategy during the post-WR3 visual pass so that the final package would remain easy to attribute and redistribute. The imported first-slice UI theme uses Kenney packs for panels, buttons, and icons, while later visual work fell back to team-authored presentation code for unit silhouettes and board dressing when a coherent local character pack was not yet ready for stable import [4]. This distinction matters: the UI art is third-party and documented, but the presentation wiring, layout logic, and gameplay-facing behaviour remain original coursework implementation.

The main consequence of this reference strategy is that related work shaped form more than function. External material helped the team avoid placeholder visuals and supported a clearer tabletop-fantasy identity, but it did not replace the need for local engineering decisions. The delivered system is therefore best understood as an original tactical vertical slice built with carefully bounded reference use, not as an assembly of imported mechanics.

2.3 Related-Work Boundary

Because this is a coursework prototype rather than an academic game-studies paper, the related-work discussion remains intentionally practical. The most relevant comparison points are genre conventions and asset-source choices, not a long survey of published games. The report therefore focuses on the references that materially affected implementation: the initial design brief, the one-pager that constrained scope, and the open-licensed asset sources that informed the final presentation direction [9, 5, 4].

Chapter 3

Requirements Engineering

The requirements for Tactical Cards were first established in WR1 and then refined as the team moved from concept definition to a stable vertical slice. The final requirement set preserves the original design intent of a readable tactical card game for teenagers, but it has been updated to reflect the actual delivered system rather than the earlier prototype-only view [9, 5].

3.1 Requirements Elicitation

Three stakeholder perspectives shaped the project requirements.

- **Teenage end users.** The game needed to be understandable within a short session, readable on screen, and strategically meaningful without deep genre knowledge.
- **The teaching team and module assessor.** The project needed to show clear software-engineering practice, including requirements work, planning, testing, documentation, and teamwork evidence.
- **The development and maintenance perspective.** The implementation needed to remain small enough to build, test, debug, and explain within the coursework period.

Early elicitation work, including the one-pager and WR1 reporting, narrowed the project toward a lightweight tactical battle rather than a large campaign game. That decision was reinforced by early meetings, which made it clear that the team needed a scope that could support both gameplay delivery and a strong reportable engineering process. As a result, the team treated the project as a compact battle-simulation product rather than a content-heavy game.

3.2 Requirements Specification

Table 3.1 presents the final requirement set used for the completed system. The list combines the original WR1 requirements with the post-WR3 UI/UX completion work so that the specification matches the delivered game.

ID	Requirement	Delivered meaning in the final build
FR-01	The game shall provide a complete front-end flow.	The player can move from title screen to battle and then to a result state with replay or return-to-menu actions.
FR-02	The game shall initialise one playable tactical battle on a small grid.	The battle scene creates a 6x6 board, places player and enemy units, and starts a valid battle loop.
FR-03	The player turn shall refresh battle resources and draw a hand of cards.	Turn start resets the playable-action budget and refreshes the runtime hand.
FR-04	The player shall be able to select a unit, choose a card, and act on a valid target only.	The HUD and board flow enforce valid ordering and surface invalid-target feedback.
FR-05	Command cards shall support movement, attack, defence, repositioning, and recovery.	The final card set includes Move, Dash, Strike, Long Strike, Guard, Push, and Recover.
FR-06	Enemy units shall execute visible and deterministic turns.	Enemy AI targets the nearest valid player and either attacks or advances accordingly.
FR-07	The game shall expose clear victory and defeat states.	Battle outcome transitions into a result panel with replay and menu actions.
FR-08	The runtime HUD shall expose the key battle state clearly.	The final HUD shows prompts, hand, energy, battle log, card availability, and world-space health bars.
NFR-01	The game shall remain suitable for teenage users.	The rules, UI flow, and feedback are intentionally kept compact and readable.
NFR-02	The project shall remain feasible within coursework scope.	The team keeps to a single-battle vertical slice instead of expanding into campaign systems.
NFR-03	The implementation shall remain testable and maintainable.	The codebase is modularised by responsibility and protected by EditMode and PlayMode tests.
NFR-04	Third-party assets and components shall be traceable.	Imported UI art is recorded in asset notices and supporting registers.
NFR-05	The project shall remain documentable and traceable as coursework evidence.	Milestones, tests, issue logs, and process records are archived under the project records tree.

Table 3.1: Final functional and non-functional requirements for Tactical Cards.

3.3 Requirements Validation

The requirements were validated through a combination of direct automated tests, PlayMode smoke checks, manual visual smoke, and issue-driven iteration. This layered approach was necessary because the project contains both deterministic game rules and presentation-heavy UI concerns. Core systems such as turn flow, card resolution, grid occupancy, enemy behaviour, and game bootstrap were validated directly with automated tests. By contrast, visual layout questions such as title readability or hand-panel occlusion required manual smoke supported by targeted regression tests.

Table 3.2 summarises how the main requirement groups were validated in the delivered project.

Requirement group	**Validation approach**	**Primary evidence**
Battle bootstrap and flow	EditMode bootstrap tests plus PlayMode smoke	‘GameRootTests‘, ‘GameFlowControllerTests‘, ‘BattleFlowPlayModeTests‘
Cards and turn loop	EditMode rule-level tests	‘TurnManagerTests‘, ‘DeckManagerTests‘, ‘CardResolverTests‘
Grid and unit state	EditMode placement, movement, damage, and occupancy tests	‘GridManagerTests‘, ‘UnitControllerTests‘, ‘TileViewTests‘
Enemy response	EditMode AI tests plus turn-loop integration checks	‘EnemyAITests‘, ‘TurnManagerTests‘
HUD and interaction feedback	EditMode UI tests plus manual smoke for final layout	‘BattleUITests‘, ‘CardButtonViewTests‘, post-WR3 smoke records
Usability refinements	Heuristic walkthrough followed by post-WR3 issue closure and smoke retest	WR2 walkthrough and post-WR3 issue log

Table 3.2: How the final requirement groups were validated.

This validation path also reflects how the requirements matured. Early heuristic feedback showed that onboarding clarity, enemy-phase readability, and battle-end presentation were still weak in the prototype. Those gaps later became explicit implementation and testing targets during the post-WR3 completion push. As a result, the final requirement set is not only an early design wish list; it is a refined specification backed by implementation and validation evidence.

Chapter 4

Development

The final delivered version of Tactical Cards is built as a deliberately small but complete vertical slice. The codebase does not attempt to solve campaign progression, procedural generation, or large content management. Instead, it focuses on one battle-ready scene with a clear runtime flow, explicit combat rules, a themed HUD, and enough architectural separation to keep the system understandable and testable [7, 3].

4.1 System Design

The runtime architecture is split by responsibility rather than by scene object alone. This keeps the main battle systems readable and makes it easier to test core behaviour directly in EditMode. Table 4.1 summarises the main component groups required by the final report brief.

Area	Key classes	Responsibility	Origin
Core flow	‘GameRoot‘, ‘GameFlowController‘, ‘GameFlowState‘, ‘TurnManager‘	Scene bootstrap, title-battle-result flow, battle session start, turn state, and win/loss resolution	Group-authored
Cards	‘CardData‘, ‘DeckManager‘, ‘CardResolver‘	Card definitions, deck lifecycle, draw/discard handling, and explicit effect resolution	Group-authored
Grid	‘GridManager‘, ‘TileView‘	Board generation, occupancy, movement, range queries, and highlight state	Group-authored
Units	‘UnitController‘, ‘EnemyAI‘	Shared unit state, damage/heal/guard behaviour, and deterministic enemy targeting and movement	Group-authored
Runtime HUD	‘BattleUI‘, ‘BattleHudSnapshot‘, ‘CardButtonView‘, ‘UnitHealthBarView‘	Battle-state presentation, runtime card views, prompts, disabled-card feedback, and world-space health bars	Group-authored
Presentation	‘UnitVisualShell‘, ‘WorldThemeResources‘, ‘UiThemeResources‘	Board dressing, colour palette, unit silhouettes, and themed UI resource binding	Group-authored wrapper over third-party art
Assets	Kenney UI sprites and icons	Runtime panels, banners, buttons, and icons for the themed HUD and menu flow	Third-party, recorded in notice file

Table 4.1: Main runtime components used by the delivered system.

The most important architectural boundary is the separation between battle rules and presentation. ‘TurnManager‘ owns turn progression, card-play budget, enemy phase execution, and battle-end state. ‘DeckManager‘ owns the draw pile, hand, and discard pile. ‘CardResolver‘ resolves card effects explicitly through readable branches instead of a large effect hierarchy. ‘GridManager‘ and ‘TileView‘ maintain board occupancy and targeting support. Together these classes form the deterministic gameplay core.

Above that core, ‘GameRoot‘ and ‘GameFlowController‘ coordinate scene setup and UI state. ‘GameRoot‘ finds or restores required scene dependencies, caches initial unit spawn snapshots, rebuilds the grid, resets the deck, registers units, and spawns reinforcement pressure when the starting enemy count is too low. ‘GameFlowController‘ then exposes the project as a complete front-end flow by building the runtime canvas, title panel, battle shell, and result panel inside the main scene. This avoided introducing unnecessary multi-scene

complexity while still delivering a recognisable game loop.

The code hierarchy also reflects maintainability concerns from the coursework brief. All runtime logic lives under ‘Assets/Scripts/‘, test code lives under ‘Assets/Tests/‘, and third-party runtime art is isolated under ‘Assets/ThirdParty/‘ and ‘Assets/Resources/UITheme/‘. This gives the final report a clear source-code hierarchy to describe and helps separate team-authored implementation from imported visual assets.

4.2 Implementation Strategics

From WR2 onward, the team treated the project as a single-battle vertical slice. That decision was not only a scope reduction; it was the main implementation strategy. The alternative would have been to broaden the game into more scenes, more content, and more systems before the core loop was stable. Instead, the team kept the battlefield, deck flow, and enemy response intentionally compact so that testing, debugging, and later UI refinement could be completed with evidence rather than guesswork [7].

This strategy remained visible in the post-WR3 milestones. Rather than pivoting into new subsystems, the team extended the same vertical slice in thin layers: first by adding title and result flow, then by replacing the prototype HUD with a panel-based runtime UI, then by modestly expanding gameplay, and finally by completing the visual and usability pass. Table 4.2 shows how the final phase stayed additive rather than disruptive.

Milestone	Focus	What it changed	Effect on final build
M7	Front-end flow shell	Added title, battle, and result states within the main scene	Turned the prototype into a complete session flow
M8	HUD refactor	Replaced immediate-mode presentation with a panel-based runtime HUD	Improved readability and interaction clarity
M9	Gameplay expansion	Added ‘Dash‘, ‘Long Strike‘, ‘Recover‘, and reinforcement pressure	Increased tactical variety without changing the core loop
M10	Finish pass and evidence packaging	Locked in automated reruns and smoke-backed stability checks	Made the slice submission-ready rather than prototype-only
M11-M13	Visual and UX completion	Added open-licensed theming, board ambience, health bars, and direct card-availability feedback	Improved perceived quality and reduced usability gaps

Table 4.2: Post-WR3 development remained incremental and vertical-slice focused.

The implementation strategy also relied on iterative refinement rather than one-pass delivery. For example, ‘BattleUI‘ was not simply decorated after being built; it was reshaped repeatedly in response to user smoke findings about prompt overlap, oversized hand panels, and missing feedback surfaces. Similarly, direct battle debug mode was rerouted through the shared ‘GameFlowController‘ path so the team would not accidentally validate a different runtime tree from the one the player actually sees. This is an example of implementation strategy shaping code structure, not just task order [3].

4.3 Simulation Design

The delivered game simulates one compact tactical encounter on a 6x6 grid. Two player-controlled units occupy the board at battle start and fight one or more enemies under a turn-based loop. The simulation is deliberately deterministic: there are no random enemy tactics, no procedural map changes, and

no branching objectives. This makes the battle state easier to understand for players and easier to validate for the development team.

At the start of each player turn, ‘TurnManager‘ refreshes the action budget and asks ‘DeckManager‘ to discard the previous hand and draw a new one. In the delivered build, the player draws three cards per turn and may spend two actions (‘maxCardPlaysPerTurn = 2‘). This keeps the turn short enough to remain readable while still allowing combinations such as move plus attack or defend plus reposition.

The fallback starter deck now contains nine runtime cards:

- three ‘Move‘ cards;
- one ‘Dash‘;
- one ‘Strike‘;
- one ‘Long Strike‘;
- one ‘Guard‘;
- one ‘Push‘;
- one ‘Recover‘.

This card set reflects the project’s final tactical goal: a small number of actions with visibly different roles. ‘CardResolver‘ keeps the effect logic explicit. Movement and dash actions resolve against tiles, strike and push actions resolve against enemy units, and guard/recover resolve against the acting unit. This decision preserved readability and helped the tests cover each branch directly.

Enemy behaviour is intentionally simple but sufficient for the final slice. ‘EnemyAI‘ uses Manhattan distance to select the nearest alive player unit. During the enemy phase, an enemy attacks if already adjacent; otherwise it steps toward the nearest target. This keeps the battle loop predictable enough for a first-time player while still creating pressure. To prevent the one-enemy starting setup from feeling too empty, ‘GameRoot‘ can spawn a reinforcement derived from the enemy template if the starting enemy count is below two. This is a good example of moderate gameplay expansion that increased pressure without destabilising the core rules.

Battle end handling is also part of the simulation design rather than only a UI detail. ‘TurnManager‘ owns the battle-end check, sets the winning team, freezes turn interaction, and produces a ‘GameResultData‘ payload. ‘GameFlowController‘ then turns that payload into the result-screen state. This separation ensures that victory and defeat remain logic-backed conditions first and presentation states second.

4.4 UI Design

The UI changed more dramatically than any other system between the early prototype and the delivered build. WR2 accepted a prototype-first interface because it was enough to prove the battle loop. By the final submission phase, however, this was no longer sufficient. The project needed a visible start screen, a readable battle HUD, a result screen, and stronger direct feedback about what the player could or could not do.

The front-end flow is now assembled at runtime by ‘GameFlowController‘. It creates a ‘ScreenSpaceOverlay‘ canvas, builds title, battle, and result panels, and keeps them inside the same main scene. The title panel includes the game brand, a short summary, a feature strip, and a clear call to action. The result panel exposes the battle outcome plus replay and menu actions. Keeping these screens in the same scene simplified implementation and testing while still satisfying the need for a complete start-to-finish session flow.

During battle, ‘BattleUI‘ builds a three-zone runtime HUD:

- a top bar for turn state, remaining energy, selection state, prompts, and action buttons;
- a left panel for unit summary, status, and the battle log;
- a bottom panel for the active hand of cards.

The runtime HUD no longer pulls ad hoc strings directly from scattered private state. Instead, it builds a ‘BattleHudSnapshot‘ containing turn labels, prompt text, selected-unit information, card states, and unit summaries. ‘CardButtonView‘ uses this snapshot to render card title, description, use hints, and disabled reasons directly in the hand. ‘UnitHealthBarView‘ projects health state back into world space so HP can be read from the board rather than only from the log.

The final UI was shaped heavily by late smoke findings. Title overlay content had to be separated into summary, feature, and button regions. The top bar had to be restacked into explicit rows because the turn label, energy, selection text, and prompt text were colliding. The hand dock had to be reduced because it obscured lower-board units. These are not superficial polish notes; they show how the UI design moved from thematic styling into functional UX refinement [3].

Visual theming was added without replacing the project’s own runtime structure. Kenney UI and icon assets were used for banners, panels, buttons, and symbols, while the board, props, and unit silhouettes remained driven by the team’s own presentation code. ‘WorldThemeResources‘ standardises board, unit, and prop colours; ‘UnitVisualShell‘ converts placeholder primitives into distinct knight-, ranger-, and spider-like silhouettes; and the ‘GridManager‘ board-presentation layer adds a tabletop base, frame, corner posts, and decorative props. The result is still a coursework-scale demo, but it now looks intentional rather than merely assembled.

Chapter 5

Test & System Evaluation

Testing became the main backbone of the project during WR3 and remained central during the post-WR3 completion push. The final validation approach did not rely on a single type of evidence. Instead, it combined direct EditMode tests for stable logic, PlayMode smoke for whole-scene readiness, and manual smoke for visual composition and overlay-heavy UI behaviour [9, 8, 3].

5.1 Test Strategy

The core strategy was *TDD-first where behaviour is deterministic, smoke-first where the result is primarily visual.* This mattered because Tactical Cards contains both rule-driven systems and presentation-heavy systems. Turn flow, deck lifecycle, movement constraints, damage handling, and battle-end state all have explicit success conditions and can be tested directly. By contrast, title-panel readability, overlay composition, and the visual relationship between the board and the hand dock cannot be reduced safely to screenshot-perfect automation in the current project setup.

The team therefore treated a result as *verified* only when it was backed by an explicit rerun record on disk. In practice this meant that a test file, smoke note, or issue closure was not treated as complete merely because the code existed. It had to be rerun or re-smoked after the relevant fix. This rule was inherited from WR3 and remained useful during the later UI and UX passes.

Table 5.1 summarises the final validation layers.

Layer	Main purpose	Representative evidence	Used for
EditMode tests	Directly verify deterministic rules and state transitions	‘TurnManagerTests‘, ‘DeckManagerTests‘, ‘CardResolverTests‘, ‘GridManagerTests‘, ‘GameFlowControllerTests‘, ‘BattleUITests‘	Core logic, flow, UI regressions
PlayMode smoke	Verify that the scene reaches a playable runtime state	‘BattleFlowPlayModeTests‘	Scene bootstrap and battle readiness
Manual smoke	Confirm readability, layout, and aesthetic stability	Post-WR3 UX, ambience, and HUD smoke records	Overlay composition and final UX judgement

Table 5.1: Validation layers used by the final project.

5.2 Test Methods

The automated suite is organised by runtime responsibility. This mirrors the source-tree structure and makes failures easier to interpret. The earlier WR3 validation freeze already covered the main prototype classes directly. Post-WR3 work then expanded that base by adding stronger UI and flow tests, including ‘GameFlowControllerTests‘ and additional ‘BattleUITests‘ around runtime HUD layout, disabled-card states, and direct-battle start behaviour.

At the latest post-WR3 HUD feedback checkpoint, the project reported:

- ‘81/81‘ passing EditMode tests;
- ‘1/1‘ passing PlayMode smoke test.

This is materially stronger than the WR3 freeze alone because the later counts include the new title-flow, HUD, and layout-regression work. Table 5.2 shows the progression from the archived WR3 validation baseline into the later post-WR3 reruns.

Checkpoint	EditMode	PlayMode	Primary focus	Evidence source
WR3 freeze (17/04/2026)	‘57/57‘	‘1/1‘	Battle-loop validation and requirements coverage freeze	WR3 execution summary
M12 ambience (24/04/2026)	‘75/75‘	‘1/1‘	World-layer presentation pass and title/result dressing	Ambience smoke record
M13 HUD feedback (24/04/2026)	‘81/81‘	‘1/1‘	HUD readability, disabled-card feedback, and layout regression closure	HUD feedback smoke record

Table 5.2: Progression from the WR3 validation baseline to the final post-WR3 reruns.

The suite now covers the main runtime concerns directly:

- battle start, battle end, and turn-loop correctness;
- draw, discard, reshuffle, and hand lifecycle;
- movement, attack, push, guard, heal, and occupancy rules;
- enemy target selection and movement decisions;
- battle HUD prompts, disabled-card reasoning, and runtime layout guards;
- title-flow and result-flow state transitions.

This is important because the final project was not only judged on whether the game happened to run during a demo. It was also judged on whether the team could show systematic validation around the delivered behaviour.

5.3 Code Coverage

The project does have an archived code-coverage baseline from the WR3 validation freeze. That archive recorded:

- ‘11‘ runtime classes in the coverage report;
- ‘74.2%‘ line coverage;
- ‘92.4%‘ method coverage.

Those figures come from the 17/04/2026 archived coverage export and are useful because they quantify the validation position at the point where the prototype battle slice was formally frozen for WR3 [8]. They also show that the suite was not limited to a few smoke assertions; it already covered most of the core runtime surface directly.

However, the report must stay honest about the current limitation: the coverage archive has not yet been regenerated after the post-WR3 expansion from the WR3 baseline to the later ‘81/81‘ EditMode position. In addition,

the current worktree shows the archived coverage folder in a dirty state. For that reason, the final report should treat the ‘74.2%‘ line-coverage figure as an archived WR3 baseline rather than claim it as the final post-polish coverage number. The correct next step before final submission is to restore or regenerate the archive and then update the chapter if a final-coursework coverage figure is required.

This distinction matters because coverage is only useful when tied to a known suite state. Reporting a stale number as if it belongs to the latest build would weaken the credibility of the report.

5.4 Evaluation

The delivered build satisfies the main goal of the project: it provides a complete and readable tactical battle loop rather than only a technical prototype. The player can begin from a title screen, enter battle, select units and cards, understand the turn state through the HUD, observe enemy response, and reach a result screen with replay or menu options. The core gameplay, runtime flow, and late-stage usability improvements are all supported by named evidence on disk.

Table 5.3 summarises the main outcome areas.

Area	Outcome	Reasoning
Playable front-end flow	Satisfied	Title, battle, and result states now exist inside the main scene and are covered by flow and smoke evidence.
Battle rules and turn loop	Satisfied	Movement, attack, guard, push, recover, draw/discard, enemy turns, and battle-end handling are directly tested.
HUD readability and action feedback	Satisfied	Energy, prompts, disabled-card reasons, and world-space health bars closed earlier readability gaps.
Visual and thematic cohesion	Satisfied	Themed UI assets, board ambience, and silhouette-based units now give the slice a coherent visual direction.
Regression protection	Satisfied	Late-stage UI and flow fixes were backed by reruns, not only by one-off manual checks.
Residual limitations	Open but non-blocking	The project remains a single-battle demo, overlay-heavy screenshot capture still needs manual smoke, and coverage needs a fresh post-polish export.

Table 5.3: Final evaluation summary for the delivered build.

The most valuable aspect of the final testing cycle is not any single number. It is the fact that the major late-stage defects were turned into explicit closure work. Title overlay collision, top-bar overlap, hand-panel occlusion, hidden card-availability feedback, and debug-flow bypass were all identified through smoke, traced to specific runtime causes, fixed in code, and then backed by rerun or hierarchy evidence. This is exactly the kind of test-and-improve loop the coursework expects [3].

The final system still has clear boundaries. It is not a large tactics game; it is a polished single-battle coursework slice. That is acceptable because the evaluation criteria reward rigour, traceability, and professionalism more than raw feature count. On that basis, the delivered build is much stronger than the early prototype: it is now a complete, demonstrable, and substantially validated software artifact rather than only a promising design idea.

Chapter 6

Evolution & Maintenance

Although Tactical Cards is a compact prototype, the final coursework submission still needed to demonstrate that the delivered system can be maintained and extended without structural rework. The team therefore treated maintainability as part of the product definition rather than as a post-hoc aspiration. This is most visible in the separation of runtime responsibilities, the use of test assemblies, and the way third-party assets are isolated from core gameplay code [9, 7].

6.1 Delivered Maintainability Measures

The codebase is structured around functional responsibility rather than scene-specific scripts. Core bootstrap and flow classes are separated from battle rules, card resolution, unit logic, grid state, UI presentation, and purely visual wrappers. This makes it possible to modify one layer without rewriting the others. For example, the post-WR3 UI completion work changed the title flow, HUD layout, disabled-card messaging, and health-bar presentation without replacing the underlying turn model or card resolver. That is a strong indication that the delivered architecture supports incremental maintenance rather than one-off prototype hacks.

Assembly definitions provide a second maintainability guardrail. Runtime code is grouped in the main ‘TacticalCards‘ assembly, while EditMode and PlayMode test suites are isolated in separate test assemblies. This separation reduces build pollution, avoids test-only dependencies leaking into player-facing code, and makes the validation boundary easier to explain. The same design also helped resolve earlier build issues where test dependencies were being pulled into non-test contexts.

Third-party assets are also contained deliberately. Imported UI art is recorded under the third-party and theme-resource folders, with a matching notice file and project-record register. This means later maintainers can update or replace the visual theme without having to disentangle it from game rules. Documentation follows the same principle: smoke records, milestone notes, issue logs, and final-report preparation files are kept under ‘doc/src/project-records/‘, so evidence generation does not depend on private memory or external chat history.

6.2 Deferred Work and Future Evolution

The most important deferred work is content expansion, not systems rescue. The delivered game does not include a campaign structure, multiple maps, progression systems, deep deck customisation, or a large enemy catalogue. Those omissions were intentional from the one-pager onward, because each of them would have multiplied test surface, UI complexity, and content production cost without improving the quality of the core coursework slice [5, 7]. In other words, the remaining work is mostly additive.

If the project were continued after assessment, three extension directions would provide the highest value. First, encounter variety could grow through additional objectives, alternate spawn patterns, and another map or two while preserving the same flow shell. Second, the presentation layer could be improved further by replacing the procedural unit silhouettes with a coherent imported CC0 character set and simple animation retargeting. Third, the validation package could be refreshed to include a new final-stage coverage snapshot once the current UI-polish branch stabilises completely. All three paths build on the current architecture rather than requiring a restart.

The milestone history supports this conclusion. Post-WR3 work already showed that the project can absorb front-end flow, HUD feedback, and theming extensions in thin slices without destabilising the entire battle loop. That is exactly the maintenance story a coursework prototype should aim to prove: not that everything was built, but that what was built can evolve in a controlled way [7].

Chapter 7

Problem & Reflections

The most useful reflection from this project is that many late-stage problems were not purely visual and not purely technical. In practice, presentation, interaction, testing, and process evidence were coupled. Once the team moved beyond the prototype stage, every UI or flow improvement exposed another layer of engineering work: raycast boundaries, runtime layout assumptions, battle-state messaging, or proof that a fix had actually held. This chapter summarises those issues and the team's response in concrete terms [9, 3].

7.1 Technical Problem

One recurring technical problem was that visual polish changed interaction behaviour. As the title overlay, themed HUD, and hand dock became more elaborate, previously hidden assumptions in the prototype started to fail. Concrete examples included title overlay content colliding vertically, the top-left prompt stack overlapping turn and energy labels, the hand dock obscuring lower units, and action availability being visible only in the battle log. Earlier in the post-WR3 cycle, selection blocking and battle-flow regressions also appeared because UI elements and game-state transitions were still too tightly coupled [3].

The important lesson was that these defects could not be handled with visual tweaking alone. Several bugs existed at the boundary between layout and logic. A panel could look harmless but still intercept clicks; a debug shortcut could speed up testing but bypass the same flow path used by the real HUD; a screenshot could show the board correctly while missing overlay elements entirely. The team therefore adopted a repeatable closure pattern: reproduce the problem in smoke, move any deterministic part of the failure into an EditMode or PlayMode regression test, fix the runtime code, and then rerun smoke for the final visual confirmation. This pattern is one of the strongest practical outcomes of the project because it turned late-stage polish from a subjective exercise into a controlled engineering loop.

Another technical limitation came from tooling. Unity MCP accelerated many inspections and runtime checks, but overlay screenshots were not consistently trustworthy for 'Screen Space Overlay' verification in this project setup. Instead of pretending the tooling was sufficient on its own, the team explicitly split responsibility: hierarchy assertions and runtime tests for structural

proof, and manual smoke for final overlay readability. That boundary is now documented rather than hidden, which makes the final validation claims more credible [3].

7.2 Project Management

From a project-management perspective, the main challenge was resisting attractive but expensive scope growth. The game concept could easily have drifted toward a larger tactics RPG with more cards, more maps, more animations, and progression systems. Instead, the team kept returning to the vertical-slice rule established earlier in the coursework: build one encounter well, then extend only when the current layer is stable. This is why post-WR3 work was broken into explicit slices such as front-end flow, HUD refactor, gameplay expansion, visual theming, ambience, and feedback completion, rather than into one vague "polish" phase.

Testing and evidence collection also became planning tools rather than just report-writing tasks. When a feature area lacked a clear validation path, that was treated as a planning warning sign. For example, purely cosmetic ideas that could not be stabilised or evidenced were deprioritised, while improvements such as disabled-card overlays, world-space health bars, and menu-flow fixes were promoted because they closed visible usability gaps and could be backed by tests or smoke evidence. This made the late project more disciplined: the team was not simply asking what looked better, but what improved the submission package in a demonstrable way.

7.3 Team Management

The team-management story is more mixed, and it is important to describe it honestly. Early collaboration relied heavily on cloud-file passing and chat-based coordination, which was fast but weak for traceability. Later, the team consolidated onto a canonical repository structure with clearer ownership and branch sequencing. That move improved accountability and made the final codebase and document trail much easier to audit, but it also meant that some historical process evidence had to be reconstructed or explicitly marked as incomplete [2, 1].

This is why several final-report support notes are phrased as claim boundaries rather than polished success stories. The repository now proves a much stronger late-stage workflow than the team had early on, but it does not magically recreate every historical task-board capture or meeting trace. Instead of inventing evidence, the team kept a transparent distinction between what is directly provable from the canonical repository, what has been retrospectively reconstructed, and what remains weaker than ideal. That choice may be less flattering than writing a smooth retrospective, but it is better software-engineering practice and more defensible in assessment terms [6].

The positive lesson is that convergence on one repository and one evidence tree materially improved collaboration. Once responsibilities, milestone notes,

issue logs, and report materials lived in the same workspace, ambiguity reduced sharply. In practical terms, the final package is stronger because the team eventually standardised its collaboration process, even if that standardisation came later than it ideally should have.

Chapter 8

Conclusion

This project delivered a complete and submission-ready tactical card battle prototype in Unity. The final build provides a coherent front-end flow, a functioning card-driven skirmish on a grid, readable battle feedback, a result state, documented open-asset use, and a validation trail that extends from requirements through tests, smoke checks, and issue closure. The system remains intentionally small, but that small scope is one of its strengths: it allowed the team to finish a real vertical slice rather than leave a larger design half-implemented.

The final report also shows that the most valuable work was not only in adding features, but in making the prototype explainable and maintainable. Post-WR3 milestones improved the title flow, battle HUD, usability feedback, and visual presentation while preserving the core turn model and strengthening the test package. Remaining limitations are mainly additive: more encounter variety, richer characters and animation, and a refreshed final-stage coverage snapshot would all improve the project, but none of them are required to understand or play the delivered game.

Overall, Tactical Cards meets the coursework goal of combining a game product with defensible software-engineering practice. It demonstrates a compact but complete tactical design, a stable implementation path, and a documented process that is strong enough to support final submission and future extension.

Reference

[1] Group 2. Canonical repo evidence, 2026. Repository workflow evidence note stored in the project records.

[2] Group 2. Collaboration history (truthful migration note), 2026. Collaboration migration note stored in the project records.

[3] Group 2. Post-wr3 m12 m13 issue log, 2026. Late-stage issue log stored in the project records.

[4] Group 2. Post-wr3 open asset register, 2026. Open-licensed asset shortlist and integration record stored in the project records.

[5] Group 2. Tactical cards one-pager, 2026. Working design note stored in the repository documentation.

[6] Group 2. Task board evidence, 2026. Task-board evidence note stored in the project records.

[7] Group 2. Wr2 implementation decisions, 2026. Decision log stored in the project records.

[8] Group 2. Wr3 execution summary, 2026. Testing execution record stored in the project records.

[9] University of Nottingham. Comp4135 coursework 1 specification, 2026. Local PDF stored under the course materials folder.

[10] University of Nottingham. Comp4135 final report template, 2026. Local PDF stored under the course materials folder.

Appendix A

User-manual

A.1 Purpose

This manual explains how to run and play the current Tactical Cards demo in the Unity Editor. The delivered coursework build is a compact single-battle prototype rather than a full campaign game, so the manual focuses on the playable battle loop and the runtime HUD.

A.2 Opening the Project

1. Open the Unity project located in `coursework/`.
2. Load `Assets/Scenes/SampleScene.unity`.
3. Press the Unity Editor `Play` button.

When the scene starts correctly, the player first sees the title screen rather than entering combat immediately.

A.3 Starting a Battle

1. Wait for the title screen to finish loading.
2. Press `Start Battle`.
3. The game transitions into the battle state and the battle HUD becomes active.

If the battle has already finished, the result screen allows the player to replay the battle or return to the title screen.

A.4 Core Interaction Flow

The intended interaction order during the player turn is:

1. select a player unit on the board;
2. select a card from the hand;
3. select a valid target tile or enemy unit if the card requires one;

4. observe the result and continue until the turn ends.

The player can press `Clear` to reset the current selection and `End Turn` to pass control to the enemy phase.

A.5 Reading the Battle HUD

A.5.1 Top Bar

The top bar provides the most important turn-state information:

- current turn state;
- remaining action budget, shown as `Energy`;
- current selection state;
- prompt text describing what the player should do next.

A.5.2 Left Panel

The left panel provides supporting battle information:

- unit summary;
- battle log;
- recent action history.

A.5.3 Bottom Hand Panel

The bottom panel displays the current hand of cards. Each card includes:

- card name;
- action type;
- effect summary such as range, damage, movement, or recovery value.

When a card cannot be used in the current state, the card shows disabled feedback directly instead of relying only on the battle log.

A.5.4 World-space Unit Feedback

Each unit displays a world-space health bar above the model. This allows the player to read board state without scanning the text log after every action.

A.6 Card Reference

The current fallback starter deck supports the following actions:

- **Move**: move the selected unit up to 2 tiles.
- **Dash**: move the selected unit up to 3 tiles.
- **Strike**: deal 2 damage to an adjacent enemy.
- **Long Strike**: deal 2 damage to an enemy within 2 tiles.
- **Guard**: reduce the next incoming hit against the selected unit.

- **Push**: deal 1 damage and push the enemy backward if possible.
- **Recover**: restore 2 health to the selected unit.

A.7 Understanding the Turn Loop

1. At the start of the player turn, the game refreshes the action budget and hand state.
2. The player performs a limited number of card-driven actions.
3. The player ends the turn.
4. Enemy units execute their actions automatically.
5. The game checks whether the battle should continue or end in victory or defeat.

Some battles may introduce reinforcement pressure during play. This is an intended part of the final tactical slice.

A.8 Ending the Battle

When the battle reaches a win or loss condition, the runtime flow changes to the result screen. The result screen provides:

- a visible battle outcome;
- replay/start-over behaviour;
- a return path to the title screen.

A.9 Troubleshooting

- If the title screen does not appear, verify that `SampleScene.unity` is the loaded scene and that the project compiled successfully.
- If a card cannot be played, check the current prompt text, selection state, and the card's disabled feedback.
- If board state becomes hard to follow, use the top-bar prompt plus the world-space health bars before relying on the battle log.

Appendix B

Meeting Minutes

B.1 Meeting Record Scope

This appendix section contains the meeting records currently preserved inside the workspace. Where a record was reconstructed from dated artifacts rather than preserved as a complete formal minute set, it remains labelled as *recovered*. This distinction should be preserved in the final report rather than hidden.

B.2 Recovered Implementation Checkpoint

Date: 15/03/2026
Type: recovered checkpoint note
Attendance: exact attendance not archived in this workspace

B.2.1 Purpose

Turn the WR1 proposal into a minimum playable prototype that could demonstrate:

- a tactical grid;
- card-driven actions;
- a visible turn loop;
- enemy response.

B.2.2 Recovered Decisions

- Use one battle scene only for the early playable version.
- Keep the board at 6x6 for the first prototype.
- Limit the starter prototype to four cards: Move, Strike, Guard, and Push.
- Use one shared ‘UnitController‘ for player and enemy units.
- Keep enemy behaviour deterministic and minimal.
- Prefer simple button-driven card selection over drag-and-drop interaction.

B.2.3 Recovered Follow-up Tasks

- Implement the first-pass script list and get the scene running.
- Refresh the architecture diagrams so they match the actual scripts.
- Capture prototype screenshots for WR2.
- Start recording validation and testing notes separately from design notes.

B.3 WR1 Kickoff Meeting

Date: 17/03/2026
Attendance: Guangjun HU, Hongshuo HU, Zehan CHEN

B.3.1 Context

This meeting was used to share initial game ideas, narrow the coursework direction, and assign the first WR1 tasks.

B.3.2 Decisions

- Confirm Tactical Cards as the project direction.
- Agree the initial role allocation.
- Set the first delivery expectations for WR1 materials.

B.3.3 Follow-up

- Prepare initial WR1 materials between 18/03/2026 and 19/03/2026.
- Carry the outcome into WR1 project direction, division of labour, and requirements sections.

B.4 WR1 Alignment Meeting

Date: 18/03/2026
Attendance: Guangjun HU, Cheng YE

B.4.1 Context

This meeting aligned the current requirement scope around a tactical card battle demo, assigned UML-related work, and standardised a lightweight minute-taking format.

B.4.2 Decisions

- Confirm the WR1 scope around a tactical card battle demo.
- Assign UML diagram production.
- Define a unified meeting-minutes format for later weeks.

B.4.3 Follow-up

- Finalise WR1 alignment by 19/03/2026.
- Reuse the agreed meeting-minutes format in later weekly cycles.

B.5 Testing and Validation Meeting

Date: 15/04/2026
Format: Online
Chair: Guangjun HU
Secretary: Zehan CHEN

B.5.1 Attendance

- Guangjun HU – present
- Hongshuo HU – present
- Cheng YE – present
- Zehan CHEN – present

B.5.2 Agenda

1. Review WR2 heuristic walkthrough findings and defect triage.
2. Confirm the test strategy and automated test scope.
3. Allocate report drafting tasks.
4. Set deadlines for remaining open bugs and final submission preparation.

B.5.3 Progress Since Last Meeting

- Confirm the WR2 closeout state on 02/04/2026 before starting the validation work.
- Apply the battle-end accuracy fix and verify it with EditMode tests.
- Create the initial test log and defect log under `doc/src/project-records/testing/`.

B.5.4 Key Discussion Points

- The high-severity battle-end gap was confirmed fixed by the EditMode smoke test.
- Onboarding clarity and enemy-phase emphasis were carried into the final polish pass and had to be closed with rerun-backed evidence before the evidence freeze.
- The team agreed to keep the report within the course template structure and not to introduce unrelated gameplay work during the validation cycle.
- GitHub pull-request review remained the canonical collaboration path for submission-facing changes.

B.5.5 Decisions Made

- Scope was limited to testing documentation, defect tracking, and report assembly for this cycle.
- All four members would contribute to the final report draft.
- The inherited usability issues had to be closed with named rerun evidence before the evidence freeze.

B.5.6 Action Items

Task	Owner	Due	Review owner
Draft LaTeX sections and tables	Cheng YE	16/04/2026	Guangjun HU
Run EditMode tests and capture results	Hongshuo HU	15/04/2026	Guangjun HU
Update defect register and meeting minutes	Zehan CHEN	16/04/2026	Cheng YE
Final integration check in Unity editor	Hongshuo HU	17/04/2026	Whole team

Table B.1: Action items agreed in the testing and validation meeting.

B.5.7 Issue Closure Status

- Onboarding clarity was later closed through the post-WR3 HUD and title-flow improvements.
- Enemy-phase emphasis was later closed through UI and prompt updates.
- Coverage completion remained pending at the time of the meeting and was finished later in the cycle.

B.5.8 Next Meeting

- Tentative date: 24/04/2026.
- Main agenda: final submission checklist, closed-bug retest review, and repository submission-readiness confirmation.

B.6 Current Gap Note

This workspace does not yet contain a stronger formally archived record for any later final sign-off meeting. If another confirmed formal meeting record exists outside the workspace, it should still be imported before submission.

Appendix C

Test Case

C.1 Code Coverage

Use the archived coverage export only after the current coverage folder has been restored or regenerated.

C.2 Test Coverage

Suggested source pool:

- `doc/src/project-records/testing/wr3-test-inventory-2026-04-17.md`
- `doc/src/project-records/testing/wr3-requirements-coverage-matrix-2026-04-17.md`
- `coursework/Assets/Tests/Editor/*.cs`
- `coursework/Assets/Tests/PlayMode/*.cs`

C.3 Test Results

Suggested source pool:

- `doc/src/project-records/testing/wr3-execution-summary-2026-04-17.md`
- `doc/src/project-records/testing/post-wr3-ambience-smoke-2026-04-24.md`
- `doc/src/project-records/testing/post-wr3-hud-feedback-smoke-2026-04-24.md`

Appendix D

Development Planning

Suggested source pool:

- `doc/src/project-records/process/wr3-execution-plan-2026-04-17.md`
- `doc/src/project-records/process/wr3-milestones-and-owners-2026-04-17.md`
- `doc/src/project-records/process/post-wr3-milestones-and-owners-2026-04-23.md`
- `doc/src/project-records/process/post-wr3-milestone-status-2026-04-23.md`
- `doc/src/project-records/process/project-implementation-timeline-gantt.drawio`

Appendix E

Version Control

Suggested source pool:

- `doc/src/project-records/process/canonical-repo-evidence-2026-04-02.md`
- `doc/src/project-records/git-pr-collaboration-policy-2026-04-02.md`
- `doc/src/project-records/collaboration-history-2026-04-02.md`
- `doc/src/project-records/pr-submit-cli-workflow.md`

Appendix F

Docs

Suggested source pool:

- `doc/src/project-records/post-wr3-open-asset-register-2026-04-23.md`
- `doc/src/project-records/process/post-wr3-m12-m13-issue-log-2026-04-24.md`
- `doc/src/project-records/process/final-report-outline-and-evidence-map-2026-0 md`